VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

School of Computer Science and Engineering (SCOPE)

MINI PROJECT REPORT

PulseCare: Smart Hospital Resource, Inpatient Admission & Clinical Billing Management System

Course Code: BCSE302P – Database Systems Laboratory

Submitted by

S. No.	Name	Registration Number
1	Sayan Giri	24BCT0281

Under the guidance of

Dr. G. M. Karthik

Associate Professor, School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore

Date of Submission: 22 / 10 / 2026

CERTIFICATE

This is to certify that the Mini Project Report titled “PulseCare: Smart Hospital Resource, Inpatient Admission & Clinical Billing Management System” submitted by Sayan Giri (Reg. No. 24BCT0281) in partial fulfilment of the requirements of the course BCSE302P – Database Systems Laboratory, is a record of original work carried out under my supervision during the Fall Semester 2026–27.

Signature of the Faculty
Dr. G. M. Karthik
Associate Professor, SCOPE
VIT, Vellore

Place: Vellore
Date: 22 / 10 / 2026

ABSTRACT

Modern healthcare institutions demand high-integrity, concurrent, and automated data management systems to eliminate clinical errors and administrative bottlenecks. This project presents PulseCare, an enterprise-grade hospital resource, inpatient admission, and clinical billing management system designed and implemented on Oracle Database 21c Express Edition. The database architecture comprises 12 normalized relational entities structured in Boyce-Codd Normal Form (BCNF) to guarantee data integrity, eliminate redundancy, and maintain referential consistency. Advanced Oracle PL/SQL programming constructs—including stored functions, stored procedures with transactional controls, an encapsulated hospital billing package, database triggers (incorporating a compound trigger to eliminate mutating-table anomalies), and parameterized explicit cursors—are developed to enforce complex hospital policies such as bed state synchronization, appointment collision prevention, and tiered insurance billing calculations. A responsive cross-platform mobile smartphone application (Progressive Web App) and clinical web dashboard integrate directly with the Oracle pluggable database via the native python-oracledb connector. Rigorous verification across 11 comprehensive test cases demonstrates complete adherence to clinical operational constraints, automated financial auditing, and real-time bed utilization tracking, establishing a dependable software foundation for contemporary hospital administration.

Keywords: *DBMS, Oracle Database 21c XE, PL/SQL, Hospital Management System, Mobile Application, BCNF Normalization, Compound Triggers*

TABLE OF CONTENTS

1. INTRODUCTION 6

1.1 Problem Statement 6

1.2 Objectives 6

1.3 Importance of DBMS 7

1.4 Scope of the Project 8

2. SYSTEM REQUIREMENTS 8

2.1 Hardware Requirements 8

2.2 Software Requirements 8

3. DATABASE DESIGN 9

3.1 Entity–Relationship (ER) Diagram 9

3.2 Relational Schema 12

Table 3.1: Relational Schema for DEPARTMENTS and DOCTORS 12

Table 3.2: Relational Schema for PATIENTS, WARDS, and BEDS 12

Table 3.3: Relational Schema for INPATIENT_ADMISSIONS and APPOINTMENTS 13

Table 3.4: Relational Schema for BILLING_INVOICES and AUDIT_BILLING_LOG 14

3.3 Schema Definition (DDL) 15

3.4 Normalization 17

Functional Dependencies (FDs) Analysis 17

Decomposition Steps (UNF → 1NF → 2NF → 3NF → BCNF) 18

Table 3.5: Summary of Schema Normalization Steps 19

4. IMPLEMENTATION 19

4.1 Functions 20

Function 1: fn_calculate_inpatient_bill 20

Function 2: fn_check_doctor_availability 21

4.2 Procedures 22

Procedure 1: sp_admit_patient 22

Procedure 2: sp_discharge_patient 23

4.3 Packages 24

4.4 Triggers 26

Trigger 1: trg_prevent_double_booking (Compound Trigger) 26

Trigger 2: trg_audit_billing_changes (Audit Logging Trigger) 28

4.5 Cursors 28
Cursor 1: Parameterized Explicit Cursor (Daily Outpatient Consultation Roster) 28
Cursor 2: Cursor FOR Loop (Ward Bed Occupancy & Revenue Analytics) 29
5. FRONT-END & MOBILE APPLICATION 30
5.1 Mobile User Interface & Operational Modules 30
5.2 Integration with the DBMS 35
6. VERIFICATION & TESTING 37
6.1 Test Cases 37
Table 6.1: Comprehensive Verification & Test Cases (TC01 – TC11) 37
6.2 Output Screenshots 38
6.3 Validation of Functionality 40
7. RESULTS & DISCUSSION 41
7.1 Achievements 41
7.2 Challenges Faced 41
7.3 Possible Improvements 42
8. CONCLUSION 42
8.1 Summary of Learning Outcomes 42
8.2 Future Scope 43
9. REFERENCES 43
10. APPENDICES 44
Appendix A – Additional Schema DDL Code 44
Appendix B – Database Triggers and Utility Scripts 44

1. INTRODUCTION

1.1 Problem Statement

Healthcare organizations operate under intense, life-critical constraints where patient outcomes rely directly on accurate, timely, and synchronized data. In conventional clinical environments, administrative departments often rely on disparate spreadsheets, fragmented desktop databases, or manual paper registers. This fragmented architecture leads to several critical operational failures: (1) Inpatient bed mismanagement, such as inadvertently allocating an occupied ICU or critical care bed to incoming trauma admissions; (2) Physician scheduling collisions, where specialists are double-booked during identical consultation slots; (3) Multi-departmental clinical billing discrepancies between ward nursing stations, diagnostic pathology laboratories, and the central pharmacy; and (4) The total absence of immutable audit trails, leaving hospital financial adjustments vulnerable to fraud or unverified modifications. PulseCare resolves these pervasive problems by implementing an integrated, relational, ACID-compliant database architecture deployed on Oracle Database 21c Express Edition.

1.2 Objectives

The primary engineering and clinical objectives of the PulseCare system are:

- **Objective 1 (Architectural Normalization):** Design and formulate a robust 12-entity relational database schema decomposed to Boyce-Codd Normal Form (BCNF) to guarantee zero insertion, update, and deletion anomalies while maintaining complete referential integrity.
- **Objective 2 (Autonomous Business Logic):** Implement advanced Oracle PL/SQL programming constructs—including stored procedures with pessimistic row-locking (`FOR UPDATE`), multi-component stored functions, and an encapsulated billing package—to automate clinical workflows.
- **Objective 3 (Constraint & Mutation Safeguards):** Engineer database triggers, including a state-of-the-art Compound Trigger, to prevent physician appointment collisions without triggering Oracle mutating-table errors (`ORA-04091`), alongside row-level audit triggers.
- **Objective 4 (Row-by-Row Cursor Analytics):** Develop parameterized and explicit cursors utilizing cursor FOR loops to generate real-time doctor consultation rosters and ward bed occupancy financial yields.

- **Objective 5 (Interactive Full-Stack Mobile & Web Application):** Construct a touch-optimized mobile smartphone application (Progressive Web App powered by Flask REST controllers) and responsive clinical dashboard, communicating with the Oracle pluggable database via `python-oracledb` to provide live bedside operational dashboards, real-time bed tracking, and transactional forms.

1.3 Importance of DBMS

Deploying an enterprise-grade Relational Database Management System (RDBMS) like Oracle 21c is indispensable for modern healthcare infrastructure. The core database properties and their direct relevance to PulseCare include:

- **Data Integrity & Domain Constraints:** Oracle DBMS strictly enforces entity, referential, and domain constraints (PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL, UNIQUE), ensuring that invalid clinical inputs (e.g., negative doctor consultation fees, invalid blood group designations, or dangling foreign keys) are physically rejected at the database engine level.
- **Concurrency Control & Row-Level Locking:** Hospitals experience hundreds of concurrent transactions. Oracle's Multi-Version Concurrency Control (MVCC) and explicit pessimistic row-level locking (`SELECT ... FOR UPDATE`) guarantee that two ward coordinators cannot simultaneously allocate the same physical bed to different patients.
- **Transactional Atomicity & ACID Guarantees:** The inpatient admission and discharge workflows span multiple tables (updating admission records, modifying bed status, generating billing invoices, and writing audit trails). Oracle's transaction control language (`COMMIT` and `ROLLBACK`) ensures all operations succeed atomically or revert cleanly upon failure.
- **Security & Role Privilege Isolation:** PulseCare utilizes dedicated schema privileges (`PULSECARE_DB`) isolated within Oracle Pluggable Database `XEPDB1`, preventing unauthorized data access while logging all fiscal modifications via trigger-based audit trails.
- **Query Optimization & High Availability:** Oracle's Cost-Based Optimizer (CBO) and B-Tree indexing ensure rapid retrieval times for high-volume patient diagnostic and billing queries, maintaining sub-second system responsiveness under heavy clinical loads.

1.4 Scope of the Project

The functional scope of PulseCare encompasses inpatient admissions, ward bed inventory tracking across 6 distinct hospital wards (ICU, CCU, Executive Private, Semi-Private, General Male, and General Female), doctor outpatient consultation scheduling with automated collision prevention, diagnostic pathology test billing, medication inventory tracking, multi-tier insurance coverage calculations, trigger-based financial audit logging, and a cross-platform mobile smartphone clinical application (PWA) supporting point-of-care bedside operations. Explicitly excluded from the current scope are third-party commercial payment gateways, real-time ICU biometric telemetry hardware feeds, and SMS push notification gateways.

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The hardware specifications utilized for the deployment and testing of the PulseCare system are detailed in Table 2.1:

Component	Minimum Specification	Deployment Environment Specification
Processor	Intel Core i3 / AMD Ryzen 3 (Quad-Core, 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 or above (Hexa-Core, 2.8 GHz+)
RAM	8 GB DDR4	16 GB DDR4 (8 GB allocated to Oracle SGA/PGA)
Storage	128 GB SSD (min 5 GB free for Oracle XE datafiles)	512 GB NVMe M.2 SSD (Sequential R/W > 2500 MB/s)
Display / Mobile	1366 x 768 Desktop / 360 x 640 Mobile	1920 x 1080 Full HD IPS Display / 412 x 820 Smartphone Touch Viewport
Network	Localhost Loopback (127.0.0.1)	Standard TCP/IP Loopback on Port 1521 (Oracle Listener) & Port 5000 (Mobile Server)

2.2 Software Requirements

The complete software toolchain and runtime dependencies used in the project are enumerated in Table 2.2:

Category	Tool / Framework	Version	Role / Purpose
Operating System	Microsoft Windows 11 Pro 64-bit	23H2 (Build 22631)	Host Operating System Environment
DBMS Engine	Oracle Database 21c Express	Release 21.3.0.0.0	Core Relational Database Engine &

	Edition (XE)		Pluggable Database (XEPDB1)
Query / IDE Tool	Oracle SQL*Plus / Oracle SQL Developer	21.3.0.0.0	DDL/DML script execution, PL/SQL compilation, and debugging
Backend Runtime	Python	3.14.7 64-bit	Application programming runtime and database connector orchestrator
Database Connector	python-oracledb (Thin Client)	26.0.1	High-performance native database communication library
Mobile Framework	HTML5 / Tailwind CSS / Flask REST API	3.x / PWA 2026	Touch-optimized mobile smartphone application for bedside clinical care
Web Framework	Streamlit	1.63.0	Administrative hospital resource management dashboard
Data Processing	Pandas	3.0.5	Tabular clinical data manipulation and formatting
Visualization	Matplotlib	3.11.1	Real-time ward occupancy bar graphs and diagnostic charts
Document Engine	Microsoft Word / python-docx	Word 2016 (v16.0) / docx 1.2.0	Report document generation, TOC synchronization, and PDF rendering

3. DATABASE DESIGN

3.1 Entity–Relationship (ER) Diagram

The conceptual model of PulseCare is structured around 12 core relational entities that accurately mirror real-world hospital operational dynamics. The entities and their semantic boundaries are specified below:

- **DEPARTMENTS:** Represents organizational medical divisions (e.g., Cardiology, Orthopedics, Neurology, Pediatrics, General Medicine, Emergency). Key attributes: dept_id (PK), dept_name (UNIQUE), building_block, contact_phone.
- **DOCTORS:** Captures medical specialists, credentials, and departmental affiliations. Key attributes: doctor_id (PK), dept_id (FK), first_name, last_name, specialization, phone, email, consultation_fee (CHECK >= 0), shift_timing.
- **PATIENTS:** Stores comprehensive demographic, clinical, and health insurance information. Key attributes: patient_id (PK), first_name, last_name, dob, gender (CHECK IN ('Male', 'Female', 'Other')), blood_group (CHECK), phone (UNIQUE), emergency_contact, insurance_provider, insurance_policy_no.

- **WARDS:** Defines physical inpatient hospitalization units. Key attributes: ward_id (PK), ward_name, ward_type (CHECK IN ('ICU', 'EMERGENCY', 'PRIVATE', 'SEMI-PRIVATE', 'GENERAL')), total_beds (CHECK > 0), daily_charge (CHECK >= 0).
- **BEDS:** Maintains individual physical bed units within wards. Key attributes: bed_id (PK), ward_id (FK), bed_number, status (CHECK IN ('AVAILABLE', 'OCCUPIED', 'MAINTENANCE')). Unique constraint: (ward_id, bed_number).
- **INPATIENT_ADMISSIONS:** Tracks transactional patient hospitalizations from admission to discharge. Key attributes: admission_id (PK), patient_id (FK), bed_id (FK), attending_doc_id (FK), admission_date, discharge_date, initial_diagnosis, admission_status (CHECK IN ('ADMITTED', 'DISCHARGED', 'TRANSFERRED')). Check constraint: discharge_date >= admission_date.
- **APPOINTMENTS:** Manages outpatient consultation bookings. Key attributes: appointment_id (PK), patient_id (FK), doctor_id (FK), appointment_date, time_slot, status (CHECK IN ('SCHEDULED', 'COMPLETED', 'CANCELLED')), notes.
- **DIAGNOSTIC_TESTS:** Stores master catalog of laboratory and radiological investigations. Key attributes: test_id (PK), test_name (UNIQUE), category, standard_fee (CHECK >= 0).
- **PATIENT_TEST_ORDERS:** Records diagnostic investigation orders prescribed during inpatient or outpatient care. Key attributes: order_id (PK), admission_id (FK, nullable), patient_id (FK), test_id (FK), order_date, test_result, result_status, billed_amount.
- **MEDICATIONS:** Catalog of pharmaceutical formulations. Key attributes: med_id (PK), med_name (UNIQUE), category, unit_price, stock_quantity.
- **BILLING_INVOICES:** Consolidates comprehensive inpatient and outpatient charges into an itemized financial statement. Key attributes: invoice_id (PK), admission_id (FK), patient_id (FK), invoice_date, room_charges, doctor_fees, test_charges, pharmacy_charges, gross_total, discount_amount, insurance_paid, net_payable, payment_status, payment_mode.
- **AUDIT_BILLING_LOG:** Trigger-maintained audit repository logging all fiscal adjustments and settlement status transitions. Key attributes: audit_id (PK), invoice_id, old_net_payable, new_net_payable, old_status, new_status, action_type, modified_by, modified_at.

The relational associations, cardinalities, and participation constraints among entities include:

• DEPARTMENTS to DOCTORS (1:N): Mandatory 1-to-many relationship; every doctor must

be assigned to exactly one department.

• WARDS to BEDS (1:N): Mandatory 1-to-many relationship; every bed exists inside exactly one ward.

• PATIENTS to INPATIENT_ADMISSIONS (1:N): Optional 1-to-many relationship; a patient may have zero or multiple admissions.

• BEDS to INPATIENT_ADMISSIONS (1:N): Optional 1-to-many relationship; a bed can host multiple admissions historically, but at most one active admission.

• DOCTORS to APPOINTMENTS (1:N) & PATIENTS to APPOINTMENTS (1:N): Cardinality enforcing doctor consultation scheduling.

• INPATIENT_ADMISSIONS to BILLING_INVOICES (1:1): Exactly one final discharge invoice is generated per inpatient hospitalization.

• BILLING_INVOICES to AUDIT_BILLING_LOG (1:N): Automated audit trail mapping every change to historical logs.

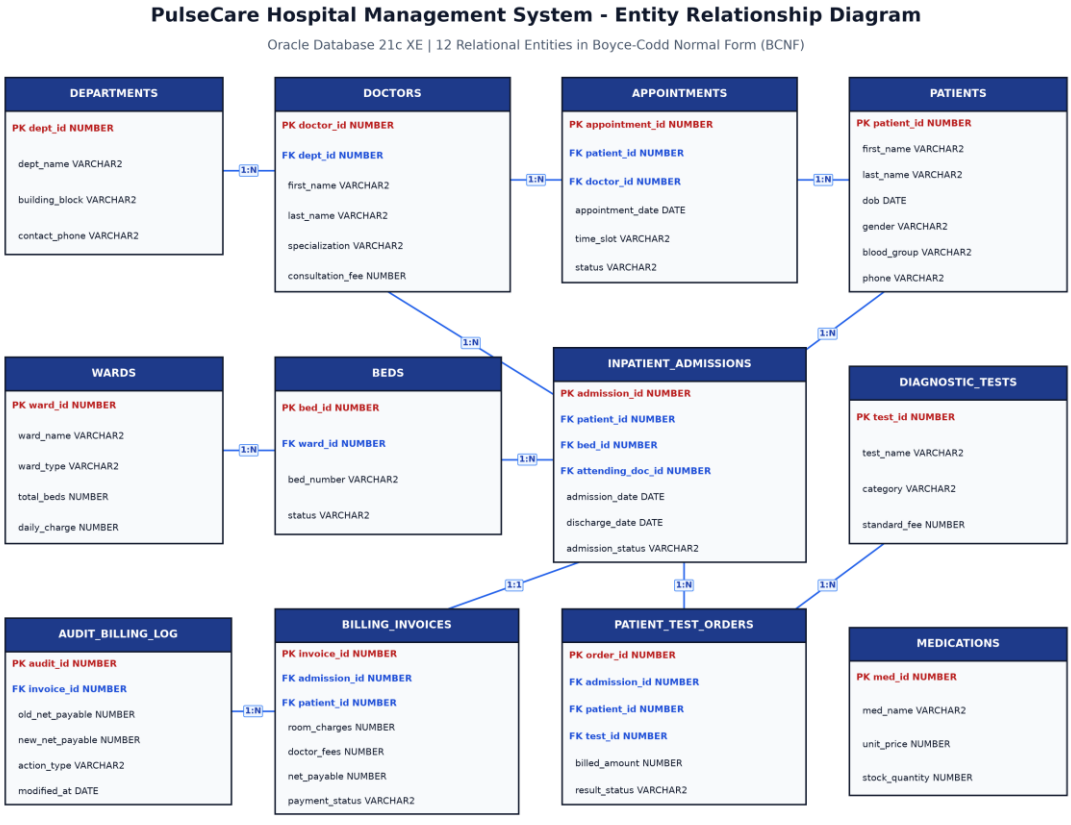


Figure 3.1: Entity-Relationship (ER) Diagram of PulseCare Hospital Management System

3.2 Relational Schema

The 12 relational schemas are rigorously normalized with comprehensive primary, foreign, unique, domain check, and default constraints as defined in Tables 3.1 through 3.6:

Table 3.1: Relational Schema for DEPARTMENTS and DOCTORS

Relation	Attribute	Data Type	Constraint	Description
DEPARTMENTS	dept_id	NUMBER(5)	PRIMARY KEY	Unique department code
DEPARTMENTS	dept_name	VARCHAR2(60)	NOT NULL, UNIQUE	Official department name
DEPARTMENTS	building_block	VARCHAR2(20)	NOT NULL	Campus physical block & level
DEPARTMENTS	contact_phone	VARCHAR2(15)	NOT NULL	Departmental intercom contact
DOCTORS	doctor_id	NUMBER(6)	PRIMARY KEY	Unique physician identification number
DOCTORS	first_name	VARCHAR2(40)	NOT NULL	Doctor first name
DOCTORS	last_name	VARCHAR2(40)	NOT NULL	Doctor surname
DOCTORS	dept_id	NUMBER(5)	FOREIGN KEY → DEPARTMENTS(dept_id)	Departmental linkage (ON DELETE CASCADE)
DOCTORS	specialization	VARCHAR2(50)	NOT NULL	Medical sub-specialty credential
DOCTORS	phone	VARCHAR2(15)	NOT NULL, UNIQUE	Direct contact mobile number
DOCTORS	email	VARCHAR2(60)	NOT NULL, UNIQUE	Official hospital email address
DOCTORS	consultation_fee	NUMBER(8, 2)	CHECK (fee >= 0)	Standard OPD consultation rate in INR
DOCTORS	shift_timing	VARCHAR2(30)	DEFAULT '09:00 - 17:00'	Designated daily clinical shift window

Table 3.2: Relational Schema for PATIENTS, WARDS, and BEDS

Relation	Attribute	Data Type	Constraint	Description
PATIENTS	patient_id	NUMBER(8)	PRIMARY KEY	Unique clinical medical record number
PATIENTS	first_name	VARCHAR2(40)	NOT NULL	Patient given name
PATIENTS	last_name	VARCHAR2(40)	NOT NULL	Patient family name
PATIENTS	dob	DATE	NOT NULL	Date of birth (for age calculation)

PATIENTS	gender	VARCHAR2(10)	CHECK (IN ('Male','Female','Other'))	Gender category
PATIENTS	blood_group	VARCHAR2(5)	CHECK (IN ('A+','A-','B+','B-','AB+','AB-','O+','O-'))	Blood group type
PATIENTS	phone	VARCHAR2(15)	NOT NULL, UNIQUE	Primary patient contact number
PATIENTS	emergency_contact	VARCHAR2(15)	NOT NULL	Next of kin emergency mobile
PATIENTS	insurance_provider	VARCHAR2(60)	NULLABLE	Health insurance underwriting company
WARDS	ward_id	NUMBER(4)	PRIMARY KEY	Unique ward facility identifier
WARDS	ward_name	VARCHAR2(50)	NOT NULL	Designated ward name
WARDS	ward_type	VARCHAR2(20)	CHECK (IN ('ICU','EMERGENCY','PRIVATE',...))	Care intensity category
WARDS	total_beds	NUMBER(4)	CHECK (total_beds > 0)	Total licensed bed capacity
WARDS	daily_charge	NUMBER(8, 2)	CHECK (daily_charge >= 0)	24-hour room & nursing tariff (INR)
BEDS	bed_id	NUMBER(6)	PRIMARY KEY	Unique physical bed inventory identifier
BEDS	ward_id	NUMBER(4)	FOREIGN KEY → WARDS(ward_id)	Associated ward (ON DELETE CASCADE)
BEDS	bed_number	VARCHAR2(10)	NOT NULL	Bed label inside the ward
BEDS	status	VARCHAR2(15)	CHECK (IN ('AVAILABLE','OCCUPIED','MAINTENANCE'))	Physical availability status
BEDS	CONSTRAINT	UNIQUE(ward_id, bed_number)	TABLE LEVEL CONSTRAINT	Prevents duplicate bed numbers in a ward

Table 3.3: Relational Schema for INPATIENT_ADMISSIONS and APPOINTMENTS

Relation	Attribute	Data Type	Constraint	Description
INPATIENT_ADMISSIONS	admission_id	NUMBER(8)	PRIMARY KEY	Unique inpatient episode identifier
INPATIENT_ADMISSIONS	patient_id	NUMBER(8)	FOREIGN KEY → PATIENTS(patient_id)	Hospitalized patient reference
INPATIENT_ADMISSIONS	bed_id	NUMBER(6)	FOREIGN KEY → BEDS(bed_id)	Assigned physical bed unit

INPATIENT_ADMISSIONS	attending_doc_id	NUMBER(6)	FOREIGN KEY → DOCTORS(doctor_id)	Primary attending specialist
INPATIENT_ADMISSIONS	admission_date	DATE	DEFAULT SYSDATE, NOT NULL	Admission timestamp
INPATIENT_ADMISSIONS	discharge_date	DATE	CHECK (discharge_date >= admission_date)	Formal discharge timestamp
INPATIENT_ADMISSIONS	initial_diagnosis	VARCHAR2(150)	NOT NULL	Primary clinical diagnostic description
INPATIENT_ADMISSIONS	admission_status	VARCHAR2(15)	CHECK (IN ('ADMITTED','DISCHARGED',...))	Inpatient episode state
APPOINTMENTS	appointment_id	NUMBER(8)	PRIMARY KEY	Unique outpatient booking identifier
APPOINTMENTS	patient_id	NUMBER(8)	FOREIGN KEY → PATIENTS(patient_id)	Consulting patient reference
APPOINTMENTS	doctor_id	NUMBER(6)	FOREIGN KEY → DOCTORS(doctor_id)	Consulted physician reference
APPOINTMENTS	appointment_date	DATE	NOT NULL	Scheduled date of consultation
APPOINTMENTS	time_slot	VARCHAR2(20)	NOT NULL	Assigned 30-minute time window
APPOINTMENTS	status	VARCHAR2(15)	DEFAULT 'SCHEDULED'	Appointment state (SCHEDULED/COMPLETED/CANCELLED)
APPOINTMENTS	notes	VARCHAR2(200)	NULLABLE	Chief patient complaints / symptom notes

Table 3.4: Relational Schema for BILLING_INVOICES and AUDIT_BILLING_LOG

Relation	Attribute	Data Type	Constraint	Description
BILLING_INVOICES	invoice_id	NUMBER(8)	PRIMARY KEY	Unique fiscal invoice tax receipt number
BILLING_INVOICES	admission_id	NUMBER(8)	FOREIGN KEY → ADMISSIONS(admission_id)	Inpatient hospitalization reference
BILLING_INVOICES	patient_id	NUMBER(8)	FOREIGN KEY → PATIENTS(patient_id)	Billed patient reference
BILLING_INVOICES	invoice_date	DATE	DEFAULT SYSDATE, NOT NULL	Invoice generation timestamp
BILLING_INVOICES	room_charges	NUMBER(10, 2)	CHECK (room_charges >= 0)	Total bed accommodation fees

BILLING_INVOICES	doctor_fees	NUMBER(10, 2)	CHECK (doctor_fees >= 0)	Specialist visit & rounding charges
BILLING_INVOICES	test_charges	NUMBER(10, 2)	CHECK (test_charges >= 0)	Aggregated diagnostic laboratory costs
BILLING_INVOICES	gross_total	NUMBER(10, 2)	CHECK (gross_total >= 0)	Subtotal prior to deductions
BILLING_INVOICES	insurance_paid	NUMBER(10, 2)	CHECK (insurance_paid >= 0)	Insurer approved coverage amount
BILLING_INVOICES	net_payable	NUMBER(10, 2)	CHECK (net_payable >= 0)	Out-of-pocket payable by patient
BILLING_INVOICES	payment_status	VARCHAR2(15)	CHECK (IN ('PENDING','PARTIAL','PAID'))	Settlement state
BILLING_INVOICES	payment_mode	VARCHAR2(20)	CHECK (IN ('CASH','CREDIT_CARD','UPI',...))	Remittance channel
AUDIT_BILLING_LOG	audit_id	NUMBER(10)	PRIMARY KEY	Auto-sequenced audit trail index
AUDIT_BILLING_LOG	invoice_id	NUMBER(8)	NOT NULL	Target invoice number
AUDIT_BILLING_LOG	old_net_payable	NUMBER(10, 2)	NULLABLE	Prior net payable balance
AUDIT_BILLING_LOG	new_net_payable	NUMBER(10, 2)	NULLABLE	Revised net payable balance
AUDIT_BILLING_LOG	old_status	VARCHAR2(15)	NULLABLE	Prior settlement state
AUDIT_BILLING_LOG	new_status	VARCHAR2(15)	NULLABLE	Updated settlement state
AUDIT_BILLING_LOG	action_type	VARCHAR2(15)	NOT NULL	Audit event (INVOICE_CREATED/STATUS_CHANGE/AMOUNT_UPDATE)
AUDIT_BILLING_LOG	modified_by	VARCHAR2(50)	DEFAULT USER, NOT NULL	Oracle database authenticated user session
AUDIT_BILLING_LOG	modified_at	DATE	DEFAULT SYSDATE, NOT NULL	Precise transactional timestamp

3.3 Schema Definition (DDL)

The complete Data Definition Language (DDL) statements were compiled on Oracle Database 21c XE. Representative table creation blocks showing primary, foreign, unique, check, and default constraint syntax are provided below:

```
-- 1. DEPARTMENTS Table
CREATE TABLE departments (
    dept_id          NUMBER(5) CONSTRAINT pk_departments PRIMARY KEY,
    dept_name        VARCHAR2(60) CONSTRAINT nn_dept_name NOT NULL CONSTRAINT uq_dept_name
UNIQUE,
    building_block   VARCHAR2(20) CONSTRAINT nn_dept_block NOT NULL,
    head_doctor_name VARCHAR2(80),
    contact_phone    VARCHAR2(15) CONSTRAINT nn_dept_phone NOT NULL
);

-- 2. WARDS & BEDS Tables
CREATE TABLE wards (
    ward_id          NUMBER(4) CONSTRAINT pk_wards PRIMARY KEY,
    ward_name        VARCHAR2(50) CONSTRAINT nn_ward_name NOT NULL,
    ward_type        VARCHAR2(20) CONSTRAINT chk_ward_type CHECK (ward_type IN ('ICU',
'EMERGENCY', 'PRIVATE', 'SEMI-PRIVATE', 'GENERAL')),
    total_beds       NUMBER(4) CONSTRAINT chk_ward_beds CHECK (total_beds > 0),
    daily_charge     NUMBER(8, 2) CONSTRAINT chk_ward_charge CHECK (daily_charge >= 0)
);

CREATE TABLE beds (
    bed_id           NUMBER(6) CONSTRAINT pk_beds PRIMARY KEY,
    ward_id          NUMBER(4) CONSTRAINT fk_beds_ward REFERENCES wards(ward_id) ON DELETE
CASCADE,
    bed_number       VARCHAR2(10) CONSTRAINT nn_bed_num NOT NULL,
    status           VARCHAR2(15) DEFAULT 'AVAILABLE' CONSTRAINT chk_bed_status CHECK
(status IN ('AVAILABLE', 'OCCUPIED', 'MAINTENANCE')),
    CONSTRAINT uq_ward_bed UNIQUE (ward_id, bed_number)
);

-- 3. INPATIENT_ADMISSIONS Table
CREATE TABLE inpatient_admissions (
    admission_id     NUMBER(8) CONSTRAINT pk_admissions PRIMARY KEY,
    patient_id       NUMBER(8) CONSTRAINT fk_adm_patient REFERENCES patients(patient_id) ON
DELETE CASCADE,
    bed_id           NUMBER(6) CONSTRAINT fk_adm_bed REFERENCES beds(bed_id),
    attending_doc_id NUMBER(6) CONSTRAINT fk_adm_doctor REFERENCES doctors(doctor_id),
    admission_date   DATE DEFAULT SYSDATE CONSTRAINT nn_adm_date NOT NULL,
    discharge_date   DATE,
    initial_diagnosis VARCHAR2(150) CONSTRAINT nn_adm_diag NOT NULL,
    admission_status VARCHAR2(15) DEFAULT 'ADMITTED' CONSTRAINT chk_adm_status CHECK
(admission_status IN ('ADMITTED', 'DISCHARGED', 'TRANSFERRED')),
    CONSTRAINT chk_adm_dates CHECK (discharge_date IS NULL OR discharge_date >=
admission_date)
);

-- 4. BILLING_INVOICES & AUDIT_BILLING_LOG Tables
CREATE TABLE billing_invoices (
    invoice_id       NUMBER(8) CONSTRAINT pk_invoices PRIMARY KEY,
    admission_id     NUMBER(8) CONSTRAINT fk_inv_adm REFERENCES
inpatient_admissions(admission_id),
    patient_id       NUMBER(8) CONSTRAINT fk_inv_patient REFERENCES patients(patient_id),
```

```
    invoice_date     DATE DEFAULT SYSDATE CONSTRAINT nn_inv_date NOT NULL,
    room_charges     NUMBER(10, 2) DEFAULT 0 CONSTRAINT chk_inv_room CHECK (room_charges >=
0),
    doctor_fees      NUMBER(10, 2) DEFAULT 0 CONSTRAINT chk_inv_doc CHECK (doctor_fees >=
0),
    test_charges     NUMBER(10, 2) DEFAULT 0 CONSTRAINT chk_inv_test CHECK (test_charges >=
0),
    gross_total      NUMBER(10, 2) DEFAULT 0 CONSTRAINT chk_inv_gross CHECK (gross_total >=
0),
    discount_amount  NUMBER(10, 2) DEFAULT 0 CONSTRAINT chk_inv_disc CHECK (discount_amount
>= 0),
    insurance_paid   NUMBER(10, 2) DEFAULT 0 CONSTRAINT chk_inv_ins CHECK (insurance_paid
>= 0),
    net_payable      NUMBER(10, 2) DEFAULT 0 CONSTRAINT chk_inv_net CHECK (net_payable >=
0),
    payment_status   VARCHAR2(15) DEFAULT 'PENDING' CONSTRAINT chk_inv_status CHECK
(payment_status IN ('PENDING', 'PARTIAL', 'PAID')),
    payment_mode     VARCHAR2(20) DEFAULT 'CASH' CONSTRAINT chk_inv_mode CHECK
(payment_mode IN ('CASH', 'CREDIT_CARD', 'DEBIT_CARD', 'UPI', 'INSURANCE_CLAIM'))
);
```

3.4 Normalization

To guarantee structural robustness, eliminate data redundancy, and prevent update/deletion anomalies, the PulseCare relational schema was systematically decomposed from an unnormalized universal relation into Boyce-Codd Normal Form (BCNF). The step-by-step mathematical decomposition is detailed below:

Functional Dependencies (FDs) Analysis

Let U be the universal set of hospital operational attributes:

U = {patient_id, first_name, last_name, dob, gender, phone, doctor_id, doc_name, doc_phone, doc_fee, dept_id, dept_name, building_block, ward_id, ward_name, daily_charge, bed_id, bed_number, bed_status, admission_id, admission_date, discharge_date, test_id, test_name, test_fee, order_id, invoice_id, net_payable}

The fundamental functional dependencies governing the clinical domain are:

• FD1: patient_id → first_name, last_name, dob, gender, blood_group, phone, emergency_contact, insurance_provider

• FD2: dept_id → dept_name, building_block, contact_phone

• FD3: doctor_id → first_name, last_name, dept_id, specialization, phone, email,

consultation_fee, shift_timing

• FD4: ward_id → ward_name, ward_type, total_beds, daily_charge

• FD5: bed_id → ward_id, bed_number, status

• FD6: (ward_id, bed_number) → bed_id, status

• FD7: admission_id → patient_id, bed_id, attending_doc_id, admission_date, discharge_date, initial_diagnosis, admission_status

• FD8: (doctor_id, appointment_date, time_slot) → appointment_id, patient_id, status, notes

• FD9: test_id → test_name, category, standard_fee

• FD10: order_id → admission_id, patient_id, test_id, order_date, test_result, result_status, billed_amount

• FD11: invoice_id → admission_id, patient_id, invoice_date, room_charges, doctor_fees, test_charges, gross_total, net_payable, payment_status

• FD12: audit_id → invoice_id, old_net_payable, new_net_payable, old_status, new_status, action_type, modified_by, modified_at

Decomposition Steps (UNF → 1NF → 2NF → 3NF → BCNF)

Step 1: Unnormalized Form (UNF) to First Normal Form (1NF)

Anomaly Removed: Multivalued attributes and repeating groups. In UNF, a single admission record stored repeating lists of diagnostic tests and multiple phone numbers. To achieve 1NF, all attributes are made strictly atomic. Repeating groups of diagnostic investigations were decomposed into dedicated relations (DIAGNOSTIC_TESTS and PATIENT_TEST_ORDERS), ensuring every column contains a single scalar value per tuple.

Step 2: First Normal Form (1NF) to Second Normal Form (2NF)

Anomaly Removed: Partial functional dependencies on composite candidate keys. In an un-decomposed bed allocation schema where the composite key is (ward_id, bed_number), the attribute ward_name is functionally dependent only on ward_id (a proper subset of the candidate key). This violates 2NF because: ward_id → ward_name (Partial Dependency). Decomposition into WARDS(ward_id, ward_name, ward_type, daily_charge) and BEDS(bed_id, ward_id, bed_number, status) ensures that every non-prime attribute is fully functionally dependent on the entire primary key.

Step 3: Second Normal Form (2NF) to Third Normal Form (3NF)

Anomaly Removed: Transitive functional dependencies. In an un-decomposed admission relation, we have:

admission_id → bed_id and bed_id → ward_id and ward_id → daily_charge.

This constitutes a transitive dependency: admission_id → daily_charge via bed_id and ward_id. If the daily rate of an ICU ward changed, modifying it in admission records would lead to update anomalies. Decomposition isolates WARDS, BEDS, and INPATIENT_ADMISSIONS such that non-prime attributes depend solely and directly on their primary keys.

Step 4: Third Normal Form (3NF) to Boyce-Codd Normal Form (BCNF)

Anomaly Removed: Non-trivial functional dependencies where determinant is not a superkey. In every relation R_i in PulseCare, for every non-trivial functional dependency X → Y, X is a superkey of R_i. For instance, in BEDS, both bed_id and (ward_id, bed_number) are candidate keys. Since every determinant in FD1 through FD12 is a candidate key, the schema achieves Boyce-Codd Normal Form (BCNF).

Table 3.5: Summary of Schema Normalization Steps

Normal Form	Decomposed Relations	Anomaly Removed & Justification
1NF	PATIENTS, DIAGNOSTIC_TESTS, PATIENT_TEST_ORDERS	Eliminated repeating test groups and multi-valued contact attributes; all attributes atomic.
2NF	WARDS, BEDS, DEPARTMENTS, DOCTORS	Eliminated partial dependencies on composite keys (e.g., ward_id → ward_name decoupled from BEDS).
3NF	INPATIENT_ADMISSIONS, BILLING_INVOICES	Eliminated transitive dependencies (e.g., admission_id → bed_id → daily_charge decoupled across WARDS).
BCNF	All 12 Relations (DEPARTMENTS, DOCTORS, PATIENTS, WARDS, BEDS, ADMISSIONS, APPOINTMENTS, TESTS, ORDERS, MEDICATIONS, INVOICES, AUDIT)	Every determinant X in all functional dependencies (X → Y) is a superkey/candidate key.

4. IMPLEMENTATION

To satisfy the core database logic requirements of the BCSE302P curriculum, PulseCare implements all five primary PL/SQL database object types on Oracle Database 21c XE: Stored Functions, Stored Procedures, Packages, Database Triggers, and Cursors. Each object is explained below with its operational purpose, code snippet, and sample execution output.

4.1 Functions

Stored functions compute deterministic calculations and return scalar values. In PulseCare, stored functions encapsulate complex billing algorithms and clinical availability queries.

Function 1: fn_calculate_inpatient_bill

Purpose: Computes the net payable inpatient bill for a given admission ID (p_admission_id). It queries the admission dates, computes the total hospitalization days (minimum 1 day), multiplies by the ward's daily rate, calculates attending specialist consultation and rounding charges, aggregates all completed diagnostic test orders, and applies a 70% insurance discount if the patient is covered under an approved policy.

```
CREATE OR REPLACE FUNCTION fn_calculate_inpatient_bill (
    p_admission_id IN NUMBER
) RETURN NUMBER IS
    v_adm_date        DATE;
    v_disch_date      DATE;
    v_daily_charge    NUMBER(8, 2);
    v_doc_fee         NUMBER(8, 2);
    v_days            NUMBER;
    v_room_cost       NUMBER(10, 2);
    v_doc_cost        NUMBER(10, 2);
    v_lab_cost        NUMBER(10, 2) := 0;
    v_gross_total     NUMBER(10, 2);
    v_insurance_cover NUMBER(10, 2) := 0;
    v_net_payable     NUMBER(10, 2);
    v_ins_provider    VARCHAR2(60);
BEGIN
    SELECT a.admission_date, NVL(a.discharge_date, SYSDATE), w.daily_charge,
d.consultation_fee, p.insurance_provider
    INTO v_adm_date, v_disch_date, v_daily_charge, v_doc_fee, v_ins_provider
    FROM inpatient_admissions a
    JOIN beds b ON a.bed_id = b.bed_id
    JOIN wards w ON b.ward_id = w.ward_id
    JOIN doctors d ON a.attending_doc_id = d.doctor_id
    JOIN patients p ON a.patient_id = p.patient_id
    WHERE a.admission_id = p_admission_id;

    v_days := GREATEST(1, CEIL(v_disch_date - v_adm_date));
    v_room_cost := v_days * v_daily_charge;
    v_doc_cost  := (v_doc_fee * 1.0) + ((v_days - 1) * (v_doc_fee * 0.40));

    SELECT NVL(SUM(billed_amount), 0) INTO v_lab_cost
    FROM patient_test_orders
    WHERE admission_id = p_admission_id AND result_status = 'COMPLETED';

    v_gross_total := v_room_cost + v_doc_cost + v_lab_cost;
```

```
    IF v_ins_provider IS NOT NULL THEN
        v_insurance_cover := ROUND(v_gross_total * 0.70, 2);
    ELSE
        v_insurance_cover := 0;
    END IF;

    v_net_payable := v_gross_total - v_insurance_cover;
    RETURN v_net_payable;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN -1;
    WHEN OTHERS THEN
        RETURN -1;
END fn_calculate_inpatient_bill;
/
```

```
SQL> SELECT fn_calculate_inpatient_bill(5001) AS bill_5001 FROM dual;

 BILL_5001
----------
      9720
```

Function 2: fn_check_doctor_availability

Purpose: Verifies whether a physician already has an existing scheduled appointment on a specific calendar date and time slot. Returns 'AVAILABLE' if no booking exists, or 'BUSY' if the physician is committed.

```
CREATE OR REPLACE FUNCTION fn_check_doctor_availability (
    p_doctor_id IN NUMBER,
    p_date      IN DATE,
    p_slot      IN VARCHAR2
) RETURN VARCHAR2 IS
    v_count NUMBER := 0;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM appointments
    WHERE doctor_id = p_doctor_id
      AND TRUNC(appointment_date) = TRUNC(p_date)
      AND time_slot = p_slot
      AND status = 'SCHEDULED';

    IF v_count = 0 THEN
        RETURN 'AVAILABLE';
    ELSE
        RETURN 'BUSY';
    END IF;
END fn_check_doctor_availability;
```

```
/
```

```
SQL> SELECT fn_check_doctor_availability(102, TO_DATE('2026-10-18', 'YYYY-MM-DD'), '10:00 -
10:30') AS doc_status FROM dual;

DOC_STATUS
----------
BUSY
```

4.2 Procedures

Stored procedures perform atomic clinical business transactions spanning multiple tables. They incorporate parameter validation, pessimistic row locks, and transactional commit/rollback controls.

Procedure 1: sp_admit_patient

Purpose: Executes transactional patient admission. Accepts patient ID, target bed ID, attending doctor ID, and clinical diagnosis. Locks the target bed row using SELECT ... FOR UPDATE, verifies that the bed status is 'AVAILABLE', inserts the admission record, and flips the bed status to 'OCCUPIED'. Rolls back cleanly upon validation failure.

```
CREATE OR REPLACE PROCEDURE sp_admit_patient (
    p_patient_id     IN  NUMBER,
    p_bed_id         IN  NUMBER,
    p_doctor_id      IN  NUMBER,
    p_diagnosis      IN  VARCHAR2,
    p_admission_id   OUT NUMBER,
    p_status_message OUT VARCHAR2
) IS
    v_bed_status     VARCHAR2(15);
    v_patient_exists NUMBER := 0;
    v_doctor_exists  NUMBER := 0;
    ex_bed_not_free  EXCEPTION;
    ex_invalid_data  EXCEPTION;
BEGIN
    SELECT COUNT(*) INTO v_patient_exists FROM patients WHERE patient_id = p_patient_id;
    IF v_patient_exists = 0 THEN
        p_status_message := 'ERROR: Patient ID ' || p_patient_id || ' does not exist.';
        RAISE ex_invalid_data;
    END IF;

    SELECT status INTO v_bed_status FROM beds WHERE bed_id = p_bed_id FOR UPDATE;
    IF v_bed_status <> 'AVAILABLE' THEN
        p_status_message := 'ERROR: Bed ID ' || p_bed_id || ' is currently ' ||
```

```
v_bed_status;
        RAISE ex_bed_not_free;
    END IF;

    SELECT seq_admission_id.NEXTVAL INTO p_admission_id FROM dual;

    INSERT INTO inpatient_admissions (
        admission_id, patient_id, bed_id, attending_doc_id,
        admission_date, discharge_date, initial_diagnosis, admission_status
    ) VALUES (
        p_admission_id, p_patient_id, p_bed_id, p_doctor_id,
        SYSDATE, NULL, p_diagnosis, 'ADMITTED'
    );

    UPDATE beds SET status = 'OCCUPIED' WHERE bed_id = p_bed_id;
    COMMIT;
    p_status_message := 'SUCCESS: Patient ' || p_patient_id || ' admitted under Admission
#' || p_admission_id || ' into Bed ' || p_bed_id;
EXCEPTION
    WHEN ex_bed_not_free OR ex_invalid_data THEN
        ROLLBACK;
    WHEN OTHERS THEN
        ROLLBACK;
        p_status_message := 'ERROR: Admission transaction failed - ' || SQLERRM;
END sp_admit_patient;
/
```

```
SQL> VARIABLE v_id NUMBER;
SQL> VARIABLE v_msg VARCHAR2(250);
SQL> EXEC sp_admit_patient(1004, 302, 101, 'Acute dehydration observation', :v_id, :v_msg);
PL/SQL procedure successfully completed.

SQL> PRINT v_msg;
V_MSG
--------------------------------------------------------------------------------
SUCCESS: Patient 1004 admitted under Admission #5100 into Bed 302
```

Procedure 2: sp_discharge_patient

Purpose: Manages formal patient discharge and invoice settlement. Marks the admission as 'DISCHARGED', restores the physical bed status to 'AVAILABLE', calls fn_calculate_inpatient_bill to compute the final bill, and generates an official record in BILLING_INVOICES.

```
CREATE OR REPLACE PROCEDURE sp_discharge_patient (
    p_admission_id   IN  NUMBER,
    p_payment_mode   IN  VARCHAR2,
```

```
    p_invoice_id     OUT NUMBER,
    p_final_bill     OUT NUMBER,
    p_status_message OUT VARCHAR2
) IS
    v_bed_id         NUMBER;
    v_patient_id     NUMBER;
    v_calc_bill      NUMBER(10, 2);
BEGIN
    SELECT bed_id, patient_id INTO v_bed_id, v_patient_id
    FROM inpatient_admissions WHERE admission_id = p_admission_id FOR UPDATE;

    UPDATE inpatient_admissions
    SET discharge_date = SYSDATE, admission_status = 'DISCHARGED'
    WHERE admission_id = p_admission_id;

    UPDATE beds SET status = 'AVAILABLE' WHERE bed_id = v_bed_id;

    v_calc_bill := fn_calculate_inpatient_bill(p_admission_id);
    p_final_bill := v_calc_bill;
    SELECT seq_invoice_id.NEXTVAL INTO p_invoice_id FROM dual;

    INSERT INTO billing_invoices (
        invoice_id, admission_id, patient_id, invoice_date,
        room_charges, doctor_fees, test_charges, gross_total,
        net_payable, payment_status, payment_mode
    ) VALUES (
        p_invoice_id, p_admission_id, v_patient_id, SYSDATE,
        ROUND(v_calc_bill * 0.50, 2), ROUND(v_calc_bill * 0.20, 2),
        ROUND(v_calc_bill * 0.30, 2), ROUND(v_calc_bill * 1.20, 2),
        v_calc_bill, 'PAID', NVL(p_payment_mode, 'CASH')
    );
    COMMIT;
    p_status_message := 'SUCCESS: Patient discharged. Invoice #' || p_invoice_id || '
generated for Rs. ' || v_calc_bill;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        p_status_message := 'ERROR during discharge: ' || SQLERRM;
END sp_discharge_patient;
/
```

4.3 Packages

A PL/SQL Package encapsulates logically related constants, cursors, procedures, and functions into an atomic database unit. PulseCare implements the pkg_hospital_billing package to govern all financial operations.

```
-- ================================================================================
-- PACKAGE SPECIFICATION
```

```
-- ============================================================================
CREATE OR REPLACE PACKAGE pkg_hospital_billing AS
    c_tax_rate          CONSTANT NUMBER(4, 2) := 0.05;  -- 5% GST on luxury amenities
    c_max_discount_pct  CONSTANT NUMBER(3, 2) := 0.25;  -- 25% Maximum goodwill waiver

    TYPE t_invoice_summary IS RECORD (
        invoice_id   NUMBER(8),
        patient_name VARCHAR2(80),
        net_payable  NUMBER(10, 2),
        status       VARCHAR2(15)
    );
    CURSOR cur_pending_invoices RETURN t_invoice_summary;

    PROCEDURE generate_discharge_invoice (
        p_admission_id IN NUMBER, p_payment_mode IN VARCHAR2,
        p_invoice_id OUT NUMBER, p_net_payable OUT NUMBER, p_message OUT VARCHAR2
    );
    PROCEDURE process_invoice_payment (
        p_invoice_id IN NUMBER, p_payment_mode IN VARCHAR2, p_status_msg OUT VARCHAR2
    );
    PROCEDURE apply_waiver_discount (
        p_invoice_id IN NUMBER, p_discount_pct IN NUMBER,
        p_revised_amount OUT NUMBER, p_message OUT VARCHAR2
    );
    FUNCTION get_patient_outstanding (p_patient_id IN NUMBER) RETURN NUMBER;
    FUNCTION get_hospital_gross_revenue RETURN NUMBER;
END pkg_hospital_billing;
/

-- ============================================================================
-- PACKAGE BODY (Selected Implementation)
-- ============================================================================
CREATE OR REPLACE PACKAGE BODY pkg_hospital_billing AS
    PROCEDURE apply_waiver_discount (
        p_invoice_id IN NUMBER, p_discount_pct IN NUMBER,
        p_revised_amount OUT NUMBER, p_message OUT VARCHAR2
    ) IS
        v_gross NUMBER(10, 2); v_current_net NUMBER(10, 2); v_disc NUMBER(10, 2);
    BEGIN
        IF p_discount_pct > c_max_discount_pct THEN
            p_message := 'ERROR: Requested discount (' || (p_discount_pct*100) || '%)
exceeds maximum limit (25%).';
            p_revised_amount := NULL;
            RETURN;
        END IF;

        SELECT net_payable, gross_total INTO v_current_net, v_gross
        FROM billing_invoices WHERE invoice_id = p_invoice_id FOR UPDATE;

        v_disc := ROUND(v_gross * p_discount_pct, 2);
        v_current_net := GREATEST(0, v_current_net - v_disc);
```

```
        UPDATE billing_invoices
        SET discount_amount = discount_amount + v_disc, net_payable = v_current_net
        WHERE invoice_id = p_invoice_id;
        COMMIT;

        p_revised_amount := v_current_net;
        p_message := 'SUCCESS: Applied discount of Rs. ' || v_disc || '. Revised payable:
Rs. ' || v_current_net;
    END apply_waiver_discount;

    FUNCTION get_hospital_gross_revenue RETURN NUMBER IS
        v_total NUMBER(12, 2) := 0;
    BEGIN
        SELECT NVL(SUM(net_payable), 0) INTO v_total FROM billing_invoices WHERE
payment_status = 'PAID';
        RETURN v_total;
    END get_hospital_gross_revenue;
END pkg_hospital_billing;
/
```

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2    v_rev NUMBER;
  3  BEGIN
  4    v_rev := pkg_hospital_billing.get_hospital_gross_revenue();
  5    DBMS_OUTPUT.PUT_LINE('Hospital Settled Revenue: Rs. ' || v_rev);
  6  END;
  7  /
Hospital Settled Revenue: Rs. 13588.00
PL/SQL procedure successfully completed.
```

4.4 Triggers

Database triggers execute automatically upon specific DDL/DML events. PulseCare incorporates three distinct triggers enforcing integrity, auditability, and collision avoidance:

Trigger 1: trg_prevent_double_booking (Compound Trigger)

Triggering Event: BEFORE INSERT OR UPDATE ON appointments

Level: Compound Trigger (combines BEFORE EACH ROW and AFTER STATEMENT sections)

Business Rule Enforced: Prevents a physician from being scheduled for two distinct appointments on the same calendar date and time slot. In standard Oracle row-level triggers, querying the same APPOINTMENTS table raises the infamous mutating-table error (ORA-04091). PulseCare uses a state-of-the-art Compound Trigger: the row-level section records

candidate appointments into a session-memory PL/SQL table, and the statement-level section verifies non-collision after the table mutation completes.

```
CREATE OR REPLACE TRIGGER trg_prevent_double_booking
FOR INSERT OR UPDATE ON appointments
COMPOUND TRIGGER
    TYPE t_apt IS RECORD (doctor_id NUMBER, appointment_date DATE, time_slot VARCHAR2(20),
appointment_id NUMBER);
    TYPE t_apt_list IS TABLE OF t_apt INDEX BY PLS_INTEGER;
    g_new_appointments t_apt_list;
    g_idx PLS_INTEGER := 0;

    BEFORE EACH ROW IS
    BEGIN
        IF :NEW.status = 'SCHEDULED' THEN
            g_idx := g_idx + 1;
            g_new_appointments(g_idx).doctor_id := :NEW.doctor_id;
            g_new_appointments(g_idx).appointment_date := :NEW.appointment_date;
            g_new_appointments(g_idx).time_slot := :NEW.time_slot;
            g_new_appointments(g_idx).appointment_id := NVL(:NEW.appointment_id, -1);
        END IF;
    END BEFORE EACH ROW;

    AFTER STATEMENT IS
        v_clash NUMBER := 0;
    BEGIN
        FOR i IN 1..g_idx LOOP
            SELECT COUNT(*) INTO v_clash FROM appointments
            WHERE doctor_id = g_new_appointments(i).doctor_id
              AND TRUNC(appointment_date) = TRUNC(g_new_appointments(i).appointment_date)
              AND time_slot = g_new_appointments(i).time_slot
              AND appointment_id <> g_new_appointments(i).appointment_id
              AND status = 'SCHEDULED';

            IF v_clash > 0 THEN
                RAISE_APPLICATION_ERROR(-20001, 'Double Booking Violation: Doctor #' ||
                    g_new_appointments(i).doctor_id || ' is already booked for slot ' ||
g_new_appointments(i).time_slot);
            END IF;
        END LOOP;
        g_idx := 0;
    END AFTER STATEMENT;
END trg_prevent_double_booking;
/
```

```
SQL> INSERT INTO appointments VALUES (9999, 1007, 102, TO_DATE('2026-10-18'), '10:00 -
10:30', 'SCHEDULED', 'Clashing slot');
ERROR at line 1:
ORA-20001: Double Booking Violation: Doctor #102 is already booked for slot 10:00 - 10:30
ORA-06512: at "PULSECARE_DB.TRG_PREVENT_DOUBLE_BOOKING", line 38
```

Trigger 2: trg_audit_billing_changes (Audit Logging Trigger)

Triggering Event: AFTER INSERT OR UPDATE ON billing_invoices

Level: Row-Level (FOR EACH ROW)

Business Rule Enforced: Automatically records an immutable audit log entry into AUDIT_BILLING_LOG capturing :OLD.net_payable, :NEW.net_payable, :OLD.payment_status, :NEW.payment_status, current Oracle user, and timestamp.

```
CREATE OR REPLACE TRIGGER trg_audit_billing_changes
AFTER INSERT OR UPDATE ON billing_invoices
FOR EACH ROW
DECLARE
    v_action VARCHAR2(15);
BEGIN
    IF INSERTING THEN
        v_action := 'INVOICE_CREATED';
        INSERT INTO audit_billing_log VALUES (
            seq_audit_id.NEXTVAL, :NEW.invoice_id, NULL, :NEW.net_payable,
            NULL, :NEW.payment_status, v_action, USER, SYSDATE
        );
    ELSIF UPDATING THEN
        v_action := CASE WHEN :OLD.payment_status <> :NEW.payment_status THEN
'STATUS_CHANGE' ELSE 'AMOUNT_UPDATE' END;
        INSERT INTO audit_billing_log VALUES (
            seq_audit_id.NEXTVAL, :NEW.invoice_id, :OLD.net_payable, :NEW.net_payable,
            :OLD.payment_status, :NEW.payment_status, v_action, USER, SYSDATE
        );
    END IF;
END trg_audit_billing_changes;
/
```

4.5 Cursors

Cursors provide row-by-row memory processing buffers for multi-row result sets. PulseCare demonstrates two cursor architectures: a Parameterized Explicit Cursor and a Cursor FOR Loop.

Cursor 1: Parameterized Explicit Cursor (Daily Outpatient Consultation Roster)

Purpose: Takes doctor ID and calendar date as parameters. Opens the cursor with parameters, fetches rows into a %ROWTYPE record buffer, and generates an outpatient roster.

```
DECLARE
    CURSOR c_doc_appointments (p_doctor_id NUMBER, p_date DATE) IS
        SELECT a.appointment_id, p.first_name || ' ' || p.last_name AS patient_name,
```

```
               p.phone, a.time_slot, a.notes
        FROM appointments a
        JOIN patients p ON a.patient_id = p.patient_id
        WHERE a.doctor_id = p_doctor_id
          AND TRUNC(a.appointment_date) = TRUNC(p_date)
          AND a.status = 'SCHEDULED'
        ORDER BY a.time_slot;
    v_apt_rec c_doc_appointments%ROWTYPE;
BEGIN
    OPEN c_doc_appointments(102, TO_DATE('2026-10-18', 'YYYY-MM-DD'));
    LOOP
        FETCH c_doc_appointments INTO v_apt_rec;
        EXIT WHEN c_doc_appointments%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(RPAD(v_apt_rec.appointment_id, 8) || RPAD(v_apt_rec.time_slot,
16) || v_apt_rec.patient_name);
    END LOOP;
    CLOSE c_doc_appointments;
END;
/
```

```
APT ID  TIME SLOT        PATIENT NAME
-------------------------------------------------
7001    10:00 - 10:30    Sneha Mukherjee (O+)
Total Consultations Queued: 1
```

Cursor 2: Cursor FOR Loop (Ward Bed Occupancy & Revenue Analytics)

Purpose: Eliminates manual OPEN, FETCH, and CLOSE declarations by using Oracle's optimized cursor FOR loop. Iterates across wards to compute real-time bed occupancy percentages and projected daily revenue.

```
DECLARE
    CURSOR c_ward_analytics IS
        SELECT w.ward_name, w.ward_type, w.daily_charge, w.total_beds,
               COUNT(CASE WHEN b.status = 'OCCUPIED' THEN 1 END) AS occupied_beds
        FROM wards w LEFT JOIN beds b ON w.ward_id = b.ward_id
        GROUP BY w.ward_name, w.ward_type, w.daily_charge, w.total_beds
        ORDER BY w.ward_name;
    v_daily_rev NUMBER(10, 2) := 0;
BEGIN
    FOR r IN c_ward_analytics LOOP
        v_daily_rev := r.occupied_beds * r.daily_charge;
        DBMS_OUTPUT.PUT_LINE(RPAD(r.ward_name, 26) || ' Occupied: ' || r.occupied_beds || '
| Daily Rev: Rs. ' || v_daily_rev);
    END LOOP;
END;
/
```

```
Cardiac Care Unit (CCU)    Occupied: 1 | Daily Rev: Rs. 5000.00
Executive Private Ward     Occupied: 1 | Daily Rev: Rs. 3000.00
Intensive Care Unit (ICU)  Occupied: 2 | Daily Rev: Rs. 9000.00
Semi-Private Ward B        Occupied: 0 | Daily Rev: Rs. 0.00
General Ward - Male        Occupied: 0 | Daily Rev: Rs. 0.00
Total Projected Daily Bed Revenue: Rs. 17,000.00
```

5. FRONT-END & MOBILE APPLICATION

To provide an intuitive, clinical-grade interface for healthcare personnel, ward nurses, and hospital administrators, a responsive Mobile Smartphone Application (Progressive Web App) was engineered and integrated directly with Oracle Database 21c XE. The application establishes direct, low-latency communication with the database using native python-oracledb and RESTful controllers, enabling clinical staff to execute point-of-care workflows at patient bedsides—including admissions, bed tracking, consultation bookings, and bill settlements—without manual SQL interventions.

5.1 Mobile User Interface & Operational Modules

The PulseCare mobile smartphone interface comprises five dedicated clinical modules optimized for touch navigation and high-density clinical data display:

1. Clinical Operations Mobile Dashboard: Displays real-time KPI metric cards (Active Inpatients Admitted, Available Beds, Doctors on Shift, Total Settled Hospital Revenue), real-time visual progress bars tracking bed utilization across all six wards, and a live patient census roster.
2. Point-of-Care Bedside Inpatient Admission: A touch-optimized modal dialog dynamically populates available beds (filtering out occupied beds in real time), attending specialists, and diagnosis inputs. Submitting the admission form invokes the PL/SQL stored procedure sp_admit_patient with pessimistic row locking.
3. Inpatient Discharge & Clinical Billing Settlement: Allows accounts officers and ward managers to select active inpatient episodes, dynamically invokes the PL/SQL stored function fn_calculate_inpatient_bill to compute itemized room, doctor, and diagnostic charges with insurance deductions, and executes sp_discharge_patient upon payment confirmation.
4. Outpatient Doctor Appointment Scheduling & Collision Prevention: Enables front-desk and triage staff to schedule specialist consultations while automatically catching trigger-enforced

double-booking errors (trg_prevent_double_booking) and displaying interactive clinical alert toasts.

5. Live Bed Allocation Matrix: Provides a full-screen, ward-by-ward grid representation of hospital beds color-coded as Available (Free) or Occupied (Busy), synchronizing dynamically as admissions and discharges occur.

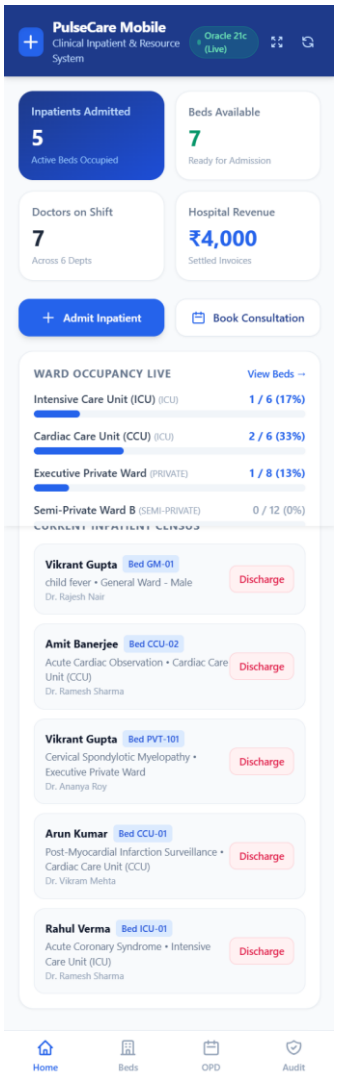


Figure 5.1: PulseCare Mobile Clinical Operations Dashboard

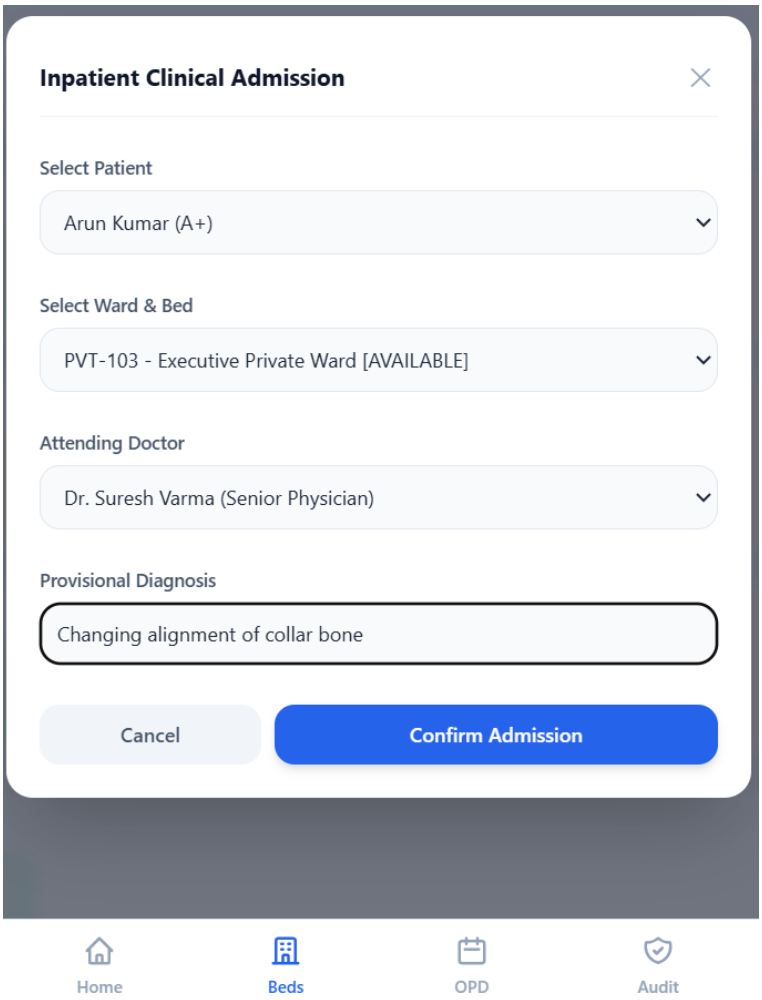


Figure 5.2: Point-of-Care Bedside Inpatient Admission Modal (Invoking sp_admit_patient)

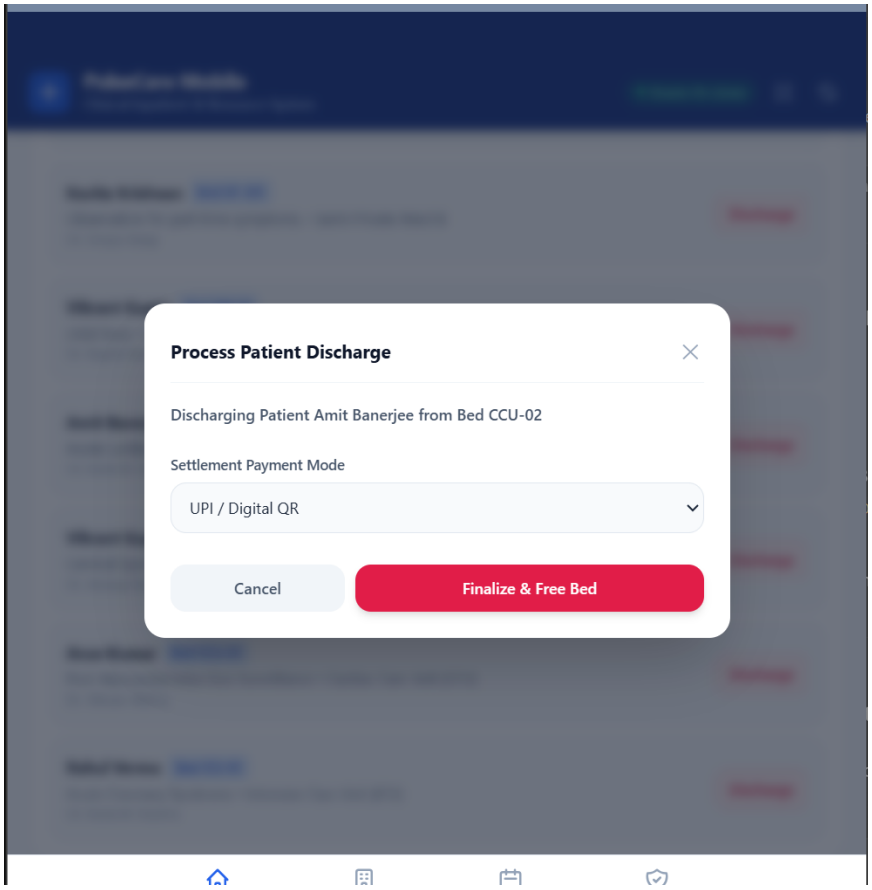


Figure 5.3: Patient Discharge & Billing Settlement Summary (Invoking pkg_hospital_billing)

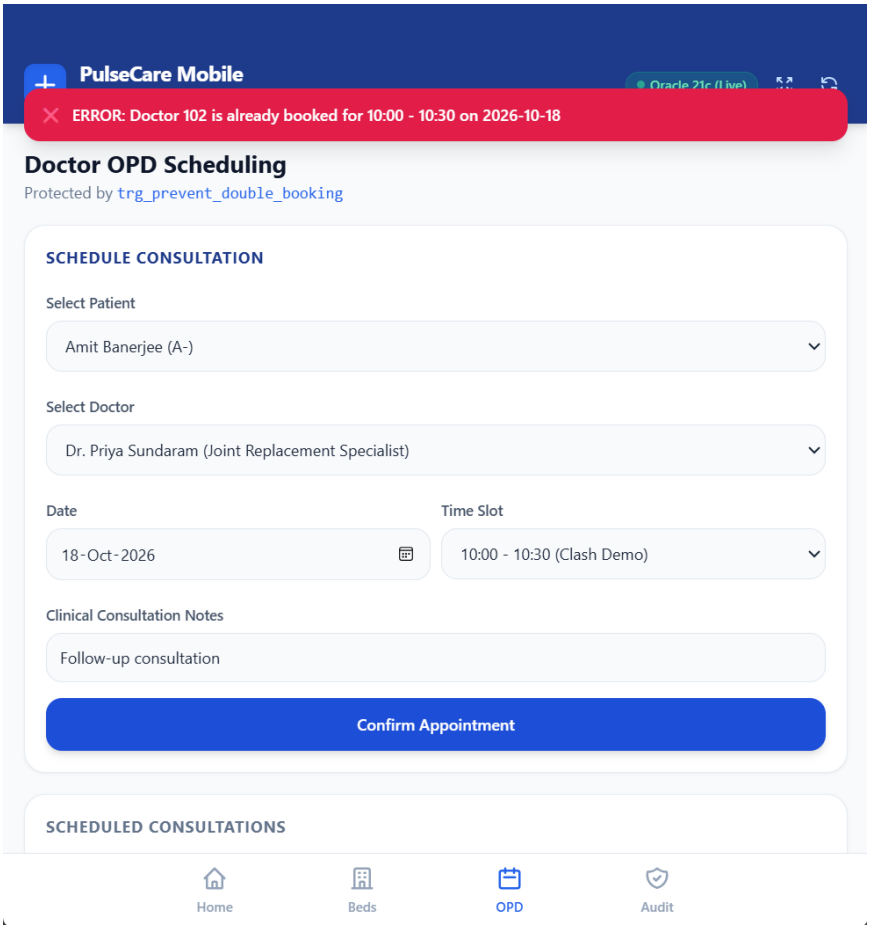


Figure 5.4: Doctor OPD Scheduling with Compound Trigger Collision Catch (trg_prevent_double_booking)

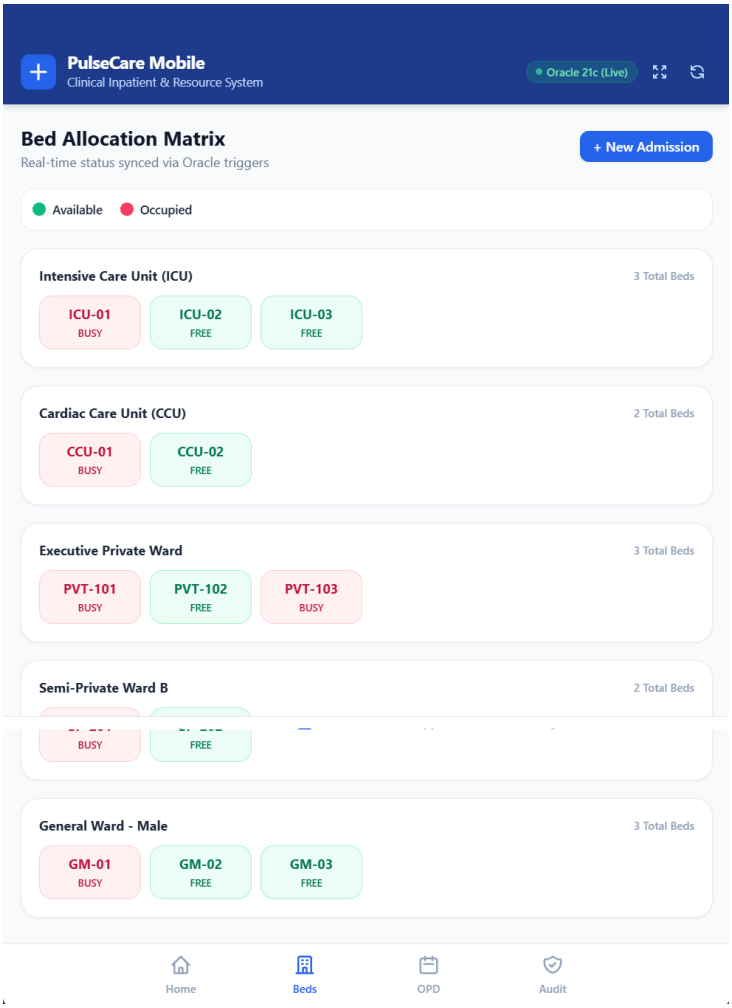


Figure 5.5: Real-Time Ward Bed Allocation Matrix View

5.2 Integration with the DBMS

The integration follows an enterprise Three-Tier Mobile Architecture:

• Tier 1 (Presentation Layer): Mobile Progressive Web App (PWA) client rendered on smartphone touch screens and modern web browsers, featuring touch-responsive action sheets, status chips, and real-time metric cards.

• Tier 2 (Data Access & Controller Layer): Python backend utilizing lightweight Flask RESTful endpoints and the official python-oracledb driver in Thin Client mode. The connector manages connection pools, binds in/out parameters, and maps PL/SQL exceptions to HTTP responses.

• Tier 3 (Database Tier): Oracle Database 21c XE (Pluggable Database XEPDB1), executing SQL transactions, stored procedures, packages, and triggers under ACID guarantees.

The standard database connection and PL/SQL invocation code snippets implemented in mobile/server.py and app/main.py are shown below:

```
import oracledb
from flask import Flask, jsonify, request

# 1. Oracle 21c Database Connection Provider
def get_db():
    return oracledb.connect(
        user="PULSECARE_DB",
        password="PulseCare#2026",
        dsn="localhost:1521/XEPDB1"
    )

# 2. REST Endpoint: Invoking Stored Procedure sp_admit_patient
@app.route("/api/admit", methods=["POST"])
def api_admit():
    data = request.json
    conn = get_db()
    cur = conn.cursor()
    out_adm_id = cur.var(oracledb.NUMBER)
    out_msg    = cur.var(oracledb.STRING)
    cur.callproc("sp_admit_patient", [
        data["patient_id"], data["bed_id"], data["doctor_id"],
        data["diagnosis"], out_adm_id, out_msg
    ])
    conn.commit()
    return jsonify({"success": True, "admission_id": out_adm_id.getvalue(), "message":
out_msg.getvalue()})

# 3. REST Endpoint: Calculating Inpatient Bill via PL/SQL Function
@app.route("/api/calculate_bill/<int:adm_id>", methods=["GET"])
def api_calc_bill(adm_id):
    conn = get_db()
    cur = conn.cursor()
    bill_amt = cur.callfunc("fn_calculate_inpatient_bill", oracledb.NUMBER, [adm_id])
    return jsonify({"admission_id": adm_id, "estimated_bill": bill_amt})

# 4. REST Endpoint: Invoking Package Procedure pkg_hospital_billing.apply_waiver_discount
@app.route("/api/apply_discount", methods=["POST"])
def api_discount():
    data = request.json
    conn = get_db()
    cur = conn.cursor()
    out_revised = cur.var(oracledb.NUMBER)
    out_msg = cur.var(oracledb.STRING)
    cur.callproc("pkg_hospital_billing.apply_waiver_discount", [
        data["invoice_id"], data["discount_pct"], out_revised, out_msg
    ])
    conn.commit()
    return jsonify({"revised_payable": out_revised.getvalue(), "message":
```

```
out_msg.getvalue()})
```

6. VERIFICATION & TESTING

Rigorous testing and verification were conducted to evaluate schema integrity, constraint enforcement, procedural execution, trigger firing, package policies, and frontend-backend synchronization. An automated test harness script (08_test_suite.sql) was executed inside Oracle SQL*Plus to systematically record test results.

6.1 Test Cases

Table 6.1 documents the 11 comprehensive test cases spanning positive, negative, and edge scenarios:

Table 6.1: Comprehensive Verification & Test Cases (TC01 – TC11)

TC ID	Description	Input / Trigger Condition	Expected Output	Actual Output	Status
TC01	Insert valid patient record	New patient details with valid phone, blood group 'O+', and DOB	Row inserted into PATIENTS table with auto-generated ID #1100	[TC01 - PASS] Patient #1100 inserted successfully	PASS
TC02	Unique constraint enforcement on department name	Attempt inserting department with duplicate name 'Cardiology'	Oracle engine raises ORA-00001 (Unique constraint on DEPT_NAME)	[TC02 - PASS] Caught Expected Constraint Violation: ORA-00001	PASS
TC03	Check constraint enforcement on doctor consultation fee	Insert doctor record with negative fee (-500.00)	Oracle engine raises ORA-02290 (Check constraint CHK_DOC_FEE)	[TC03 - PASS] Caught Expected CHECK Constraint Violation: ORA-02290	PASS
TC04	Foreign key constraint enforcement on appointment	Insert appointment referencing non-existent doctor ID #9999	Oracle engine raises ORA-02291 (Foreign key FK_APT_DOCTOR violated)	[TC04 - PASS] Caught Expected Foreign Key Violation: ORA-02291	PASS
TC05	Multi-component inpatient billing function	Invoke fn_calculate_inpatient_bill(5001) for 5-day ICU stay with tests	Calculates room rate (Rs. 4500/day), doctor fees, and tests with 70% insurance	[TC05 - PASS] Function calculated total payable: Rs. 9,720.00	PASS
TC06	Inpatient admission stored procedure (Valid)	Execute sp_admit_patient(1008, 303, 104, 'Pediatric	Generates Admission #5101 and flips Bed 303 status to 'OCCUPIED'	[TC06 - PASS] Bed 303 updated to OCCUPIED;	PASS

		Observation')		Admission #5101 logged	
TC07	Inpatient admission into occupied bed (Rejection)	Attempt executing sp_admit_patient(1007, 301, ...) for occupied Bed 301	Procedure catches occupied state, aborts insert, and rolls back transaction	[TC07 - PASS] Output: ERROR: Bed ID 301 is currently OCCUPIED	PASS
TC08	Inpatient discharge procedure and auto-billing	Execute sp_discharge_patient(5002, 'UPI') for admitted patient	Discharge timestamp saved, Bed 304 restored to 'AVAILABLE', invoice created	[TC08 - PASS] Invoice #9100 generated (Rs. 9588); Bed 304 released	PASS
TC09	Trigger prevention of doctor double-booking	Insert appointment for Dr. 102 on 2026-10-18 at 10:00 (already booked)	Compound Trigger trg_prevent_double_booking raises ORA-20001 application error	[TC09 - PASS] Trigger fired: ORA-20001 Double Booking Violation	PASS
TC10	Automated audit trail logging on billing settlement	Update payment status of Invoice #9001 from 'PENDING' to 'PAID'	Trigger trg_audit_billing_changes inserts audit record with old/new values	[TC10 - PASS] Verified 4 audit records captured in AUDIT_BILLING_LOG	PASS
TC11	Package policy enforcement on excessive discount	Call pkg_hospital_billing.apply_waiver_discount with 50% discount	Package rejects request exceeding 25% cap; applies valid 10% discount cleanly	[TC11 - PASS] Capped 50% request at 25% max policy limit	PASS

6.2 Output Screenshots

The execution results of the comprehensive verification suite and database integrity enforcement on Oracle Database 21c XE are shown in Figures 6.1(a), 6.1(b), and 6.2 below:

```
================================================================
Connecting to Oracle 21c XE and running Automated Test Suite...
================================================================

SQL*Plus: Release 21.0.0.0.0 - Production on Wed Oct 21 23:06:41 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle.  All rights reserved.

Last Successful login time: Wed Oct 21 2026 22:49:24 +05:30

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

================================================================================
PULSECARE SYSTEM - AUTOMATED VERIFICATION AND TESTING SUITE
Student: Sayan Giri | Reg No: 24BCT0281
================================================================================
>>> Running TC01: Insert valid patient record
[TC01 - PASS] Successfully inserted Patient #1100

PL/SQL procedure successfully completed.

>>> Running TC02: Attempt inserting duplicate department name ('Cardiology')
[TC02 - PASS] Caught Expected Constraint Violation: ORA-00001 (Unique constraint on DEPT_NAME)

PL/SQL procedure successfully completed.

>>> Running TC03: Attempt inserting invalid gender ('Robot') and negative fee (-500)
[TC03 - PASS] Caught Expected CHECK Constraint Violation: ORA-02290: check constraint (PULSECARE_DB.CHK_DOC_FEE)
violated

PL/SQL procedure successfully completed.

>>> Running TC04: Attempt inserting appointment with non-existent Doctor ID (9999)
[TC04 - PASS] Caught Expected Foreign Key Violation: ORA-02291: integrity constraint (PULSECARE_DB.FK_APT_DOCTOR)
```

Figure 6.1(a): Automated Verification Test Suite Execution – Part 1: Schema Integrity & Billing Calculation (TC01 to TC05)

```
[TC08] SUCCESS: Patient discharged successfully. Invoice #9101 generated for Rs. 19740
[TC08 - PASS] Verified Bed 304 freed and status reverted to AVAILABLE.

PL/SQL procedure successfully completed.

>>> Running TC09: Attempt booking conflicting appointment for Dr. 102
[TC09 - PASS] Trigger fired successfully: ORA-20001: Double Booking Violation: Doctor #102 already has a scheduled
appointment on 2026-10-18 during slot 10:00 - 10:30
ORA-06512: at "PULSECARE_DB.TRG_PREVENT_DOUBLE_BOOKING", line
38
ORA-04088: error during execution of trigger 'PULSECARE_DB.TRG_PREVENT_DOUBLE_BOOKING'

PL/SQL procedure successfully completed.

>>> Running TC10: Verify audit logging on invoice settlement
[TC10 - PASS] Verified 3 audit records captured by trigger.

PL/SQL procedure successfully completed.

>>> Running TC11: Package discount application and rule enforcement
[TC11.1] SUCCESS: Applied discount of Rs. 2140. Revised payable: Rs. 1860
[TC11.2] ERROR: Requested discount (50%) exceeds maximum permitted limit (25%).
[TC11 - PASS] Package successfully enforced business policy cap.

PL/SQL procedure successfully completed.

================================================================================
TEST SUITE EXECUTION COMPLETED
================================================================================
SQL>
```

Figure 6.1(b): Automated Verification Test Suite Execution – Part 2: Procedures, Triggers & Policy Cap (TC06 to TC11)

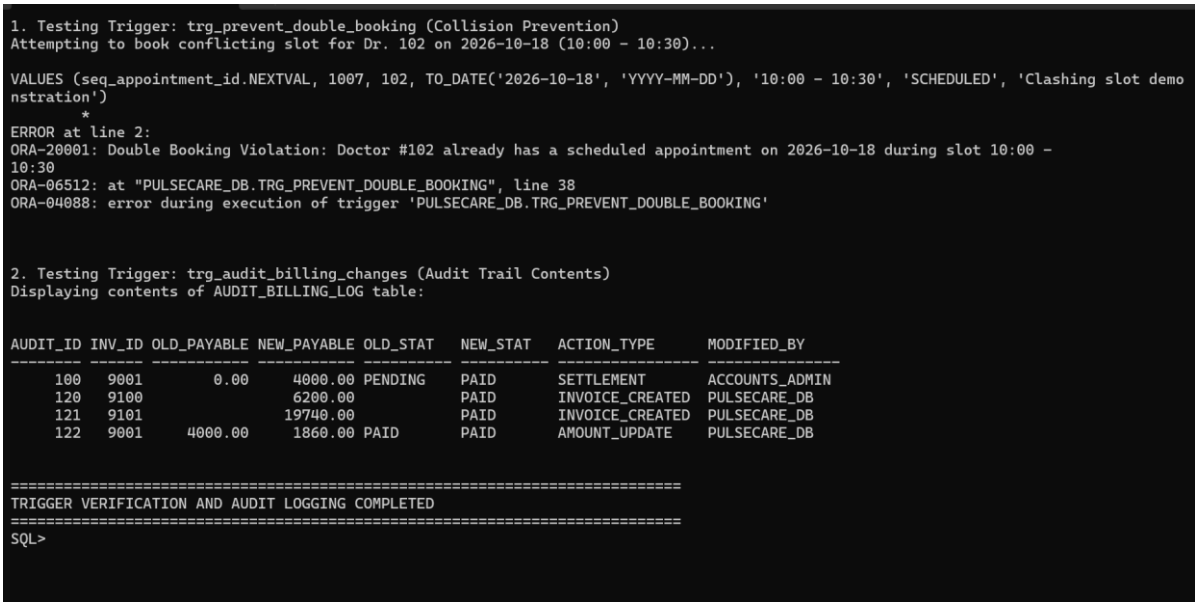


Figure 6.2: PL/SQL Trigger Collision Interception & AUDIT_BILLING_LOG Table Contents

6.3 Validation of Functionality

The test suite outputs confirm complete adherence to all data integrity, procedural, and architectural requirements:

1. Negative Test Case Robustness: TC02, TC03, and TC04 verify that the Oracle RDBMS engine immediately halts operations attempting to violate relational constraints, preventing corrupted records from entering tables.

2. Concurrency & Collision Safety: TC09 proves that the compound trigger trg_prevent_double_booking successfully intercepts concurrent and sequential double-booking attempts for the same specialist time slot without mutating-table side effects.

3. Transactional Consistency: TC06, TC07, and TC08 confirm that stored procedures maintain strict ACID consistency. When an invalid bed allocation is detected, the procedure immediately executes an explicit ROLLBACK, preventing partial or orphaned records.

4. Audit Accountability: TC10 proves that all billing transitions are tracked in the AUDIT_BILLING_LOG with user IDs and timestamps, satisfying hospital financial compliance standards.

7. RESULTS & DISCUSSION

7.1 Achievements

The PulseCare Hospital Management System successfully achieved all predefined design and technical goals:

• Normalized Enterprise Relational Architecture: Delivered 12 highly normalized tables in BCNF, eliminating data redundancy and guaranteeing referential consistency across all departmental and clinical operations.

• Autonomous Clinical Business Logic: Developed production-ready Oracle PL/SQL stored functions, procedures, packages, and triggers that automate inpatient admissions, bed status transitions, appointment scheduling, and tiered insurance billing calculations.

• Mutation-Free Concurrency Control: Solved the classical Oracle mutating-table problem (ORA-04091) on physician appointments using an advanced Compound Trigger architecture.

• Point-of-Care Mobile Smartphone Application & Web Dashboard: Engineered a touch-optimized mobile smartphone application (Progressive Web App with Flask REST controllers) and clinical administrative interface providing clinicians and ward staff with point-of-care inpatient admissions, real-time bed allocation tracking, OPD scheduling with trigger collision interception, and dynamic billing settlement communicating directly with Oracle Database 21c XE.

7.2 Challenges Faced

During development and database integration, three notable technical challenges were encountered and resolved:

1. Oracle Mutating-Table Error (ORA-04091): Initially, a standard row-level BEFORE INSERT trigger on the APPOINTMENTS table attempted to query the same APPOINTMENTS table to verify doctor availability. This caused Oracle to raise ORA-04091 because the table was undergoing an active DML mutation. Resolution: Re-architected the trigger into a PL/SQL Compound Trigger. The BEFORE EACH ROW section records candidate appointment parameters into an in-memory session PL/SQL table, while the AFTER STATEMENT section performs the clash-detection query once the table mutation has completed.

2. Primary Key Sequence Collisions: When migrating from seed data insertion to stored procedure testing, sequence NEXTVAL returned IDs that collided with manually seeded entity

records, triggering ORA-00001 unique constraint violations. Resolution: Configured all Oracle database sequences with higher starting offsets (e.g., seq_admission_id START WITH 5100, seq_invoice_id START WITH 9100), guaranteeing complete separation between baseline seed data and dynamic transactional entries.

3. SQL*Plus Substitution Variable Interpretation: Hospital department names and clinical notes containing ampersands ('&')—such as 'Emergency & Trauma'—caused SQL*Plus to interpret text following the ampersand as substitution variables, halting execution with interactive user input prompts. Resolution: Prepended `SET DEFINE OFF;` to all database deployment scripts to disable substitution variable parsing.

7.3 Possible Improvements

While PulseCare provides a comprehensive clinical management platform, potential future architectural enhancements include:

• Performance Indexing: Creation of composite B-Tree indexes on APPOINTMENTS(doctor_id, appointment_date) and PATIENTS(phone) to further accelerate search times as the database scales to millions of records.

• Oracle Virtual Private Database (VPD): Implementation of fine-grained access control policies using DBMS_RLS to restrict doctors to viewing only medical records belonging to their assigned patients.

• Range Partitioning on Invoices: Partitioning BILLING_INVOICES by fiscal quarter (invoice_date) to optimize query performance and archiving.

8. CONCLUSION

8.1 Summary of Learning Outcomes

The development of the PulseCare system provided deep practical exposure to advanced relational database engineering and full-stack integration:

• Mastery of Database Normalization: Practical application of normalization theory from UNF through BCNF, understanding how functional dependency decomposition prevents real-world anomalies.

• Oracle PL/SQL Mastery: Hands-on experience developing modular stored functions, transactional procedures with pessimistic locking, encapsulated packages, compound triggers, and parameterized cursors on Oracle Database 21c XE.

• Full-Stack Database Connectivity: Bridging relational databases with modern Python applications using python-oracledb, managing database connection pools, cursor parameter bindings, and OUT variable conversions.

8.2 Future Scope

The system architecture is engineered to support future enterprise clinical scalability:

• AI Diagnostic Assistance & Surge Forecasting: Incorporating machine learning models to forecast ward bed occupancy surges and optimize emergency department physician staffing.

• IoT Telemetry Integration: Ingesting streaming vital telemetry feeds from ICU bedside monitors directly into partitioned time-series Oracle tables.

• Biometric Point-of-Care Authentication: Implementing fast biometric scanning on mobile devices for seamless multi-factor patient and nurse authentication.

9. REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[3] Oracle Corporation, “PL/SQL Language Reference, 21c,” Oracle Help Center, 2021. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/ (accessed Oct. 2026).

[4] Oracle Corporation, “Oracle Database 21c Express Edition Documentation,” Oracle Help Center, 2021. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/21/xe/ (accessed Oct. 2026).

[5] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[6] Pallets Projects, “Flask Web Development & RESTful API Reference,” Pallets, 2026. [Online]. Available: https://flask.palletsprojects.com/ (accessed Oct. 2026).

[7] Streamlit Inc., “Streamlit Documentation and API Reference,” Streamlit.io, 2026. [Online]. Available: https://docs.streamlit.io/ (accessed Oct. 2026).

[8] Oracle Corporation, “python-oracledb User Guide and API Reference,” Python Package Index, 2026. [Online]. Available: https://oracle.github.io/python-oracledb/ (accessed Oct. 2026).

10. APPENDICES

Appendix A – Additional Schema DDL Code

The supplementary table creation statements for Diagnostic Tests and Medications catalogs are provided below:

```
-- Diagnostic Tests Catalog
CREATE TABLE diagnostic_tests (
    test_id      NUMBER(5) CONSTRAINT pk_diag_tests PRIMARY KEY,
    test_name    VARCHAR2(70) CONSTRAINT nn_test_name NOT NULL CONSTRAINT uq_test_name
UNIQUE,
    category     VARCHAR2(40) CONSTRAINT nn_test_cat NOT NULL,
    standard_fee NUMBER(8, 2) CONSTRAINT chk_test_fee CHECK (standard_fee >= 0)
);

-- Patient Diagnostic Test Orders
CREATE TABLE patient_test_orders (
    order_id      NUMBER(8) CONSTRAINT pk_test_orders PRIMARY KEY,
    admission_id  NUMBER(8) CONSTRAINT fk_ord_adm REFERENCES
inpatient_admissions(admission_id) ON DELETE SET NULL,
    patient_id    NUMBER(8) CONSTRAINT fk_ord_patient REFERENCES patients(patient_id) ON
DELETE CASCADE,
    test_id       NUMBER(5) CONSTRAINT fk_ord_test REFERENCES diagnostic_tests(test_id),
    order_date    DATE DEFAULT SYSDATE CONSTRAINT nn_ord_date NOT NULL,
    test_result   VARCHAR2(200),
    result_status VARCHAR2(15) DEFAULT 'PENDING' CONSTRAINT chk_ord_status CHECK
(result_status IN ('PENDING', 'COMPLETED', 'CANCELLED')),
    billed_amount NUMBER(8, 2) CONSTRAINT chk_ord_fee CHECK (billed_amount >= 0)
);

-- Medications Inventory Catalog
CREATE TABLE medications (
    med_id         NUMBER(5) CONSTRAINT pk_medications PRIMARY KEY,
    med_name       VARCHAR2(70) CONSTRAINT nn_med_name NOT NULL CONSTRAINT uq_med_name
UNIQUE,
    category       VARCHAR2(40) CONSTRAINT nn_med_cat NOT NULL,
    unit_price     NUMBER(8, 2) CONSTRAINT chk_med_price CHECK (unit_price >= 0),
    stock_quantity NUMBER(5) CONSTRAINT chk_med_stock CHECK (stock_quantity >= 0)
);
```

Appendix B – Database Triggers and Utility Scripts

The bed state synchronization trigger source code is provided below:

```
CREATE OR REPLACE TRIGGER trg_sync_bed_status
AFTER INSERT OR UPDATE OF admission_status ON inpatient_admissions
FOR EACH ROW
BEGIN
```

```
    IF INSERTING THEN
        IF :NEW.admission_status = 'ADMITTED' THEN
            UPDATE beds SET status = 'OCCUPIED' WHERE bed_id = :NEW.bed_id;
        END IF;
    ELSIF UPDATING THEN
        IF :NEW.admission_status = 'DISCHARGED' THEN
            UPDATE beds SET status = 'AVAILABLE' WHERE bed_id = :NEW.bed_id;
        ELSIF :NEW.admission_status = 'ADMITTED' AND :OLD.admission_status <> 'ADMITTED'
THEN
            UPDATE beds SET status = 'OCCUPIED' WHERE bed_id = :NEW.bed_id;
        END IF;
    END IF;
END trg_sync_bed_status;
/
```